SQL Lecture — Databases, DDL, and DML

1. What is a Database?

Before writing SQL, students must understand **what a database is**.

A **database** is a system used to **store and organize data**.

Think of a database like a **digital filing cabinet**.

Inside the cabinet we store **tables**.

Example table:

id	name	age	email
1	Ahmad	22	ahmad@gmail.com
2	Sara	21	sara@gmail.com

Important ideas:

Concept	Meaning
Table	collection of data
Row	one record
Column	attribute of data

2. What is SQL?

SQL stands for:

Structured Query Language

It is the language used to **communicate with databases**.

Using SQL we can:

• create tables
• insert data
• retrieve data
• update data
• delete data

3. Types of SQL Commands

SQL commands are divided into categories.

Category	Purpose
DDL	Define database structure
DML	Work with data
DCL	Control permissions

Today we will start with **DDL**.

4. DDL — Data Definition Language

DDL commands are used to **define the structure of the database**.

Think of DDL as **building the database structure**.

Important commands:

```
None
CREATE
ALTER
TRUNCATE
DROP
```

5. CREATE DATABASE

First we create a database.

Syntax

```SQL
CREATE DATABASE database_name;
```

Example

```SQL
CREATE DATABASE test;
```

Explanation:

This command tells MySQL:

> Create a new database called **test**

6. Select the Database

Now we tell MySQL to use the database.

```SQL
USE test;
```

Explanation:

```None
USE test
```

Now all queries will run **inside the test database**.

7. CREATE TABLE

Now we create a table.

Syntax

```SQL
CREATE TABLE table_name (
 column datatype constraints
);
```

Example

```SQL
CREATE TABLE students (
 id INT PRIMARY KEY,
 name VARCHAR(100),
 age INT,
 email VARCHAR(150)
);
```

Explanation

CREATE TABLE students

We create a table called **students**.

id INT PRIMARY KEY

```
None
id
```

column name

```
None
INT
```

stores numbers

```
None
PRIMARY KEY
```

unique identifier for each row.

Example:

id	name
1	Ahmad
2	Sara

No duplicates allowed.

name VARCHAR(100)

Stores text values.

Example:

```
None
Ahmad
Sara
Mohammed
```

age INT

Stores numbers.

Example:

```
None
18
22
35
```

email VARCHAR(150)

Stores email addresses.

Example:

```
None
ahmad@gmail.com
sara@yahoo.com
```

8. ALTER TABLE

ALTER is used to **modify a table structure**.

Example: add a column.

```SQL
ALTER TABLE students
ADD grade VARCHAR(20);
```

Now the table structure becomes:

| id | name | age | email | grade |

9. TRUNCATE TABLE

Removes all rows but keeps the table.

```SQL
TRUNCATE TABLE students;
```

Before:

id	name
1	Ahmad
2	Sara

After:

Table still exists but **empty**.

---

10. DROP TABLE

Deletes the table completely.

```sql
DROP TABLE students;
```

Now the table **no longer exists**.

---

11. Now Move to DML

Once students understand structure, we work with **data**.

INSERT INTO

Adds data to a table.

Syntax

```sql
INSERT INTO table_name (columns)
VALUES (values);
```

Example

```sql
INSERT INTO students (id, name, age, email)
VALUES (1,'Ahmad',22,'ahmad@gmail.com');
```

Insert another row:

```sql
INSERT INTO students (id, name, age, email)
VALUES (2,'Sara',21,'sara@gmail.com');
```

SELECT

Retrieve data.

Syntax

```SQL
SELECT columns
FROM table;
```

Example:

```SQL
SELECT * FROM students;
```

`*` means **all columns**.

Example:

```SQL
SELECT name, age
FROM students;
```

Result:

name	age
Ahmad	22
Sara	21

WHERE Clause

Filters rows.

Example:

```sql
SELECT *
FROM students
WHERE age > 21;
```

UPDATE

Modify existing data.

```sql
UPDATE students
SET age = 23
WHERE id = 1;
```

Important rule:

Without WHERE **all rows update**.

DELETE

Remove rows.

```sql
DELETE FROM students
WHERE id = 2;
```

---

12. After That — Explore Real Database

Now that students understand SQL basics, switch to the **classicmodels database**.

```sql
USE classicmodels;
```

Example query:

```sql
SELECT * FROM customers;
```

Example:

```sql
SELECT customerName, country
FROM customers;
```

Example:

```sql
SELECT productName, buyPrice
FROM products
WHERE buyPrice > 100;
```

Now students are working with **real business data**.

---

13. Questions for Students

Ask them:

1

What is the difference between

```
DELETE
TRUNCATE
DROP
```

2

What happens if we run

```sql
UPDATE students
SET age = 30;
```

3

Why do we use **PRIMARY KEY**?

---

14. Perfect Live Coding Flow

Run queries in this order.

```sql
CREATE DATABASE test;

USE test;

CREATE TABLE students (
 id INT PRIMARY KEY,
 name VARCHAR(100),
 age INT,
 email VARCHAR(150)
);

INSERT INTO students VALUES (1,'Ahmad',22,'ahmad@gmail.com');
INSERT INTO students VALUES (2,'Sara',21,'sara@gmail.com');

SELECT * FROM students;

UPDATE students
SET age = 23
```

```
WHERE id = 1;

DELETE FROM students
WHERE id = 2;
```

`ORDER BY` in Detail

What is `ORDER BY`?

The `ORDER BY` clause is used to **sort the result** of a `SELECT` query.

This means that after SQL gets the rows from the table, it can arrange them:

- from **smallest to largest**
- from **largest to smallest**
- from **A to Z**
- from **Z to A**

So `ORDER BY` does **not change the data inside the table**.
It only changes **how the result is displayed**.

Basic Syntax

```sql
SELECT column1, column2

FROM table_name

ORDER BY column_name;
```

Default sorting is **ascending (ASC)**.

Sort Customers Alphabetically

```
use classicmodels ;
SELECT customerName ,country from customers
ORDER BY customerName ASC
```

sult Grid | Filter Rows: | Export: | Wrap Cell Content:

customerName	country
Alpha Cognac	France
American Souvenirs Inc	USA
Amica Models & Co.	Italy
ANG Resellers	Spain
Anna's Decorations, Ltd	Australia
Anton Designs, Ltd.	Spain
Asian Shopping Network, Co	Singapore
Asian Treasures, Inc.	Ireland
Atelier graphique	France
Australian Collectables, Ltd	Australia
Australian Collectors, Co.	Australia
Australian Gift Network, Co	Australia
Auto Associés & Cie.	France
Auto Canal+ Petit	France
Auto-Moto Classics Inc.	USA
AV Stores, Co.	UK
Baane Mini Imports	Norway
Bavarian Collectables Imports, Co.	Germany
BG&E Collectables	Switzerland
Blauer See Auto, Co.	Germany

```sql
SELECT customerName, country

FROM customers

ORDER BY customerName ASC;
```

SELECT DISTINCT

What DISTINCT Does

DISTINCT is used to **return only unique values** from a column.

It removes **duplicate rows** from the query result.

Important idea for students:

> DISTINCT does not change the table data.
> It only removes duplicate values **in the query result**.

Basic Syntax

```sql
SELECT DISTINCT column_name

FROM table_name;
```

`LIKE` Operator (Pattern Searching)

What `LIKE` Does

The `LIKE` operator is used in the `WHERE` clause to **search for specific patterns inside text columns**.

It is commonly used when we don't know the exact value but we know **part of the text**.

For example:

- names starting with **A**
- emails containing **gmail**
- products containing **Ford**

Basic Syntax

```SQL
SELECT column_name
FROM table_name
WHERE column_name LIKE pattern;
```

Wildcard Characters

`LIKE` works using **wildcards**.

wildcard	meaning
`%`	any number of characters
`_`	exactly one character

Example 1 — Customers Whose Name Starts with "A"

Table: `customers`

Scenario:

> Marketing wants to see customers whose company name starts with **A**.

```sql
SELECT customerName
FROM customers
WHERE customerName LIKE 'A%';
```

Explanation:

```none
A% → starts with A
```

Result example:

customerName
Alpha Cognac
American Souvenirs Inc

Example 2 — Customers Containing "Mini"

Scenario:

> The company wants to find customers whose name contains the word **Mini**.

```sql
SELECT customerName
FROM customers
WHERE customerName LIKE '%Mini%';
```

Explanation:

```
%Mini%
```

means:

```
anything + Mini + anything
```

Result example:

customerName
Mini Wheels Co.

Example 3 — Employees Whose First Name Starts with "G"

Table: `employees`

Scenario:

> HR wants to find employees whose first name starts with **G**.

```sql
SELECT firstName, lastName
FROM employees
WHERE firstName LIKE 'G%';
```

Result example:

firstName	lastName
George	Vanauf
Gerard	Bondur
Gerard	Hernandez

Example 4 — Emails Ending with a Domain

Scenario:

> The company wants to find employees whose email ends with **classicmodelcars.com**.

```sql
SELECT firstName, lastName, email
FROM employees
WHERE email LIKE '%classicmodelcars.com';
```

Explanation:

```
%classicmodelcars.com
```

means:

```
None
any text before the domain
```

Example 5 — Product Names Containing "Ford"

Table: `products`

Scenario:

> The company wants to list products related to **Ford cars**.

```sql
SELECT productName
FROM products
WHERE productName LIKE '%Ford%';
```

Result example:

productName
1948 Porsche Type 356 Roadster
1957 Ford Thunderbird

Using _ (Single Character Wildcard)

_ represents **exactly one character**.

Example:

```sql
SELECT firstName
FROM employees
```

```
WHERE firstName LIKE 'G____';
```

Explanation:

```
None
G____
```

means:

- starts with G
- followed by exactly **4 characters**

Example match:

```
None
Gerar
```

Combining `LIKE` with `ORDER BY`

Scenario:

> Show employees whose names start with **G**, sorted alphabetically.

```sql
SELECT firstName, lastName
FROM employees
WHERE firstName LIKE 'G%'
ORDER BY firstName ASC;
```

Visual Explanation

If the pattern is:

```
None
LIKE 'A%'
```

It matches:

```
None
Alpha
Apple
Alex
Amanda
```

If the pattern is:

```
None
LIKE '%son'
```

It matches:

```
None
Johnson
Anderson
Jackson
```

Common Student Mistakes

Missing quotes

Wrong:

```
SQL
WHERE customerName LIKE A%
```

Correct:

```sql
WHERE customerName LIKE 'A%'
```

Confusing % and _

pattern	meaning
`A%`	starts with A
`%A`	ends with A
`%A%`	contains A

Mini Exercises for Students

Exercise 1

Find employees whose first name starts with **L**.

```sql
SELECT firstName, lastName
FROM employees
WHERE firstName LIKE 'L%';
```

Exercise 2

Find products containing the word **Classic**.

```sql
SELECT productName
FROM products
WHERE productName LIKE '%Classic%';
```

Exercise 3

Find customers whose name ends with **Co.**

```sql
SELECT customerName
FROM customers
WHERE customerName LIKE '%Co.';
```

Short Definition for Students

You can summarize it like this:

> `LIKE` is used to search for patterns in text columns using wildcard characters such as `%` and `_`.

If you want, I can also prepare **one very clear visual diagram** that explains the difference between:

```
LIKE 'A%'
LIKE '%A'
LIKE '%A%'
LIKE '_A%'
```

Students usually understand `LIKE` instantly when they see this diagram.